

CleanNotes（清记）前端开发文档 v1.3.0

本文档基于 v1.3.0 实际代码反向工程编写，所有内容均源自源码实现。

1. 项目结构

```
v1.3.0/
├── index.html           # 入口HTML: <html lang="zh-CN">, viewport cover
├── package.json         # 依赖版本清单
├── vite.config.ts       # Vite 构建配置
├── public/
│   ├── vite.svg        # Favicon
│   └── readme.txt       # 构建时从 README.md 自动复制
├── src/
│   ├── main.ts          # 应用入口: createApp → Pinia → Router → mount
│   ├── App.vue          # 根组件: 认证、桌面布局、路由出口
│   ├── style.css        # 全局样式: 4套主题（浅色/暗黑/ZURU/腾讯蓝）
│   ├── types/
│   │   └── index.ts     # 所有 TypeScript 类型定义
│   ├── router/
│   │   └── index.ts     # Hash 模式路由: PC 10条 + H5 7条 + 守卫
│   ├── stores/          # Pinia 状态管理（8个Store）
│   │   ├── auth.ts      # 认证: 手机号+密码+验证码
│   │   ├── task.ts      # 任务: CRUD + 热力图 + 远程变更合并
│   │   ├── todo.ts      # 待办事项: CRUD + 转任务
│   │   ├── memo.ts      # 备忘录: CRUD + 置顶 + 拖拽排序
│   │   ├── growth.ts    # 养成系统: XP/等级/成就/连续天数
│   │   ├── ai.ts        # AI助手: 流式对话 + Function Calling
│   │   ├── weeklyReport.ts # 周报: HTML生成 + AI总结
│   │   └── timer.ts     # 计时器: 上下班倒计时
│   ├── composables/    # Vue3 Composables
│   │   ├── useSync.ts   # 30秒增量同步循环
│   │   ├── useTheme.ts  # 5主题模式管理
│   │   ├── useGrowthIntegration.ts # 任务完成→XP/成就 事件桥接
│   │   ├── useCompletionCelebration.ts # 全部完成任务庆祝检测
│   │   └── useH5Data.ts  # H5移动端 直连Supabase CRUD
│   ├── services/       # 数据层服务
│   │   ├── storage.ts   # 存储适配器路由（local/Supabase）
│   │   ├── local.ts     # localStorage 读写实现
│   │   ├── supabase.ts  # Supabase 云端读写实现
│   │   ├── hybrid.ts    # 本地+云端混合同步引擎
│   │   ├── syncLog.ts   # 同步日志管理
│   │   ├── auth.ts      # 认证服务（Supabase用户表）
│   │   ├── memoStorage.ts # 备忘录存储服务
│   │   ├── todoStorage.ts # 待办存储服务
│   │   ├── growthStorage.ts # 养成系统存储服务
│   │   └── weeklyReportStorage.ts # 周报存储服务
│   ├── extensions/     # Tiptap 富文本编辑器扩展（13个）
│   │   ├── SlashCommand.ts # / 指令面板
│   │   ├── EmojiExtension.ts # : emoji选择器
│   │   ├── MemoMention.ts # [[ 备忘录引用
│   │   ├── DoubleBracketLinker.ts # [[ 方括号链接（备用方案）
│   │   ├── MermaidExtension.ts # Mermaid图表节点（VueNodeView）
│   │   ├── MindmapExtension.ts # 思维导图节点（VueNodeView）
│   │   ├── CalloutExtension.ts # 提示块
│   │   └── ToggleExtension.ts # 折叠/展开块
```

FileAttachmentExtension.ts	# 文件附件内联节点
DragHandle.ts	# 块级拖拽排序
MarkdownInputRules.ts	# Markdown快捷键输入规则
TrailingNode.ts	# 文档尾部空段落守卫
TaskItemStylePlugin.ts	# 任务列表项样式注入
utils/	# 工具函数
time.ts	# 时间戳工具（UTC/本地转换）
crypto.ts	# 密码哈希/校验（Web Crypto PBKDF2）
device.ts	# 设备检测（移动端/强制桌面）
markdownExport.ts	# HTML→Markdown 导出转换器
components/	# 组件库（31个Vue组件）
views/	# 页面视图（11个+ H5子目录）
layouts/	# 布局组件
H5Layout.vue	# H5移动端底部Tab布局

各目录职责

目录	职责
src/	应用源码根目录
src/stores/	Pinia状态管理，每个Store对应一个业务领域
src/services/	数据持久化层，实现 local/Supabase 双存储适配器
src/composables/	Vue3 Composition API 可复用逻辑
src/extensions/	Tiptap编辑器自定义扩展（ProseMirror Plugin）
src/utils/	纯工具函数（时间、加密、设备检测、导出）
src/components/	可复用Vue组件
src/views/	路由页面组件
src/layouts/	布局组件（H5移动端底部导航）
src/router/	路由配置+导航守卫
src/types/	TypeScript类型定义（单一文件集中管理）

2. 技术栈版本

技术	版本	说明
Vue	^3.5.13	Composition API + <script setup>
TypeScript	~5.7.2	严格模式（vue-tsc 编译检查）
Vite	^6.2.0	构建工具，支持HMR
Pinia	^3.0.2	状态管理（Setup Style 语法）
TailwindCSS	^4.1.0	原子化CSS，通过 @tailwindcss/vite 插件集成
Vue Router	^4.5.0	Hash模式路由
Tiptap	^3.26.1	富文本编辑器（核心+19个扩展包）
@tiptap/vue-3	^3.26.1	Tiptap Vue3 适配
DOMPurify	^3.4.9	HTML净化（XSS防护）

技术	版本	说明
Mermaid	^11.16.0	图表渲染（流程图/时序图等）
markmap-*	^0.18.x	思维导图（markmap-lib + markmap-view）
html2canvas-pro	^2.2.1	HTML→Canvas（周报导出）
marked	^18.0.5	Markdown解析
@tauri-apps/api	^2.3.0	Tauri桌面端API
lunar-javascript	^1.7.7	农历日历

Tiptap 扩展包清单（19个官方扩展）

@tiptap/starter-kit	# 基础套件
@tiptap/extension-mention + @tiptap/suggestion	# 提及/建议
@tiptap/extension-table + table-row/header/cell	# 表格系统
@tiptap/extension-task-list + task-item	# 任务清单
@tiptap/extension-image	# 图片
@tiptap/extension-link	# 链接
@tiptap/extension-code-block	# 代码块
@tiptap/extension-blockquote	# 引用
@tiptap/extension-bullet-list	# 无序列表
@tiptap/extension-ordered-list	# 有序列表
@tiptap/extension-list-item	# 列表项
@tiptap/extension-horizontal-rule	# 分割线
@tiptap/extension-placeholder	# 占位符
@tiptap/extension-highlight	# 高亮
@tiptap/extension-underline	# 下划线
@tiptap/extension-text-align	# 文本对齐
@tiptap/extension-text-style + color	# 文本样式/颜色
@tiptap/extension-dropcursor	# 拖放光标
tiptap-extension-resize-image	# 图片缩放

3. 应用启动流程

3.1 main.ts 初始化步骤

```
// src/main.ts
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import router from './router'
import App from './App.vue'
import './style.css'

const app = createApp(App)
app.use(createPinia()) // 步骤1: 安装 Pinia
app.use(router)        // 步骤2: 安装路由
app.mount('#app')      // 步骤3: 挂载到 #app
```

初始化顺序简洁明确：**Pinia → Router → Mount**。全局样式 `style.css` 在模块导入时立即生效，包含 TailwindCSS 和 4 套 CSS 变量主题。

3.2 App.vue 生命周期（onMounted）

```

auth.init()
├── 同步恢复 session (hasSession = true → 避免登录页闪烁)
├── 异步拉取 Supabase 最新用户信息
│   ├── 成功 → user.value = found, updateLastLogin()
│   └── 网络不可用 → 保持 session 恢复的数据

if (auth.isAuthenticated && userId):
├── switchUser(userId)           # 切换存储空间
├── runTimeMigration()           # v5: 旧时间戳→UTC ISO转换
├── recoverSparseData(userId)    # 从云端恢复丢失数据
├── taskStore.load()             # 加载本地任务
├── growthStore.load()          # 加载养成数据
├── aiStore.load()               # 加载AI消息
├── useGrowthIntegration()       # 建立 Task→Growth 事件桥
├── H5重定向检查                 # sessionStorage 'h5_redirect'
├── 路由跳转逻辑:
│   ├── 移动设备 → /h5/tasks
│   ├── PC端无路由或从登录来 → /home
│   └── PC端刷新 → 保持当前页面
├── mergeFromCloud() (后台异步)
└── 有变更 → taskStore.reload()

else:
└── router.push('/login')

```

3.3 认证恢复与数据加载时序

[同步阶段] session恢复 → isAuthenticated = true → 避免页面闪烁
 [并行加载] taskStore.load() | growthStore.load() | aiStore.load()
 [异步后台] mergeFromCloud() (不阻塞UI渲染)
 [事件桥接] useGrowthIntegration() (建立task→growth回调)

3.4 全局键盘快捷键

- Ctrl+Shift+D: 打开诊断页面 /__diag

3.5 模板结构

```

<template>
  <!-- 已认证 + 非H5路由 → PC端侧栏布局 -->
  <div v-if="showApp && !isH5Route" class="app-shell">
    <AppSidebar :is-online :sync-status :last-sync-text :sync-logs @logout />
    <main class="app-main">
      <router-view />
    </main>
    <XpToast />           <!-- XP获得弹出提示 -->
    <BackToTop />         <!-- 回到顶部按钮 -->
    <ConfirmDialog />     <!-- 重新激活任务确认弹窗 -->
  </div>
  <!-- H5路由 / 未认证 → 全屏路由视图 -->
  <router-view v-else />
</template>

```

4. 路由设计

4.1 PC端路由表（10条）

路径	Name	组件（懒加载）	meta
/login	login	@/views/LoginView.vue	{ public: true }
/	home	@/views/HomeView.vue	—
/tasks	tasks	@/views/TasksView.vue	—
/todos	todos	@/views/ToDoView.vue	—
/memos	memos	@/views/MemoView.vue	—
/ai	ai	@/views/AiView.vue	—
/settings	settings	@/views/SettingsView.vue	—
/spirit	spirit	@/views/SpiritView.vue	—
/reports	reports	@/views/ReportsView.vue	—
/__diag	diag	@/views/DiagView.vue	{ public: true }

所有路由使用 懒加载 `() => import(...)`，历史模式为 `createWebHashHistory()`（Hash模式）。

4.2 H5端路由表（7条）

父路由 `/h5` → 布局组件 `@/layouts/H5Layout.vue`

路径	Name	组件
<code>/h5 (redirect→tasks)</code>	—	H5Layout
<code>/h5/tasks</code>	<code>h5-tasks</code>	<code>@/views/h5/H5TaskList.vue</code>
<code>/h5/tasks/new</code>	<code>h5-task-new</code>	<code>@/views/h5/H5TaskEdit.vue</code>
<code>/h5/tasks/:id</code>	<code>h5-task-edit</code>	<code>@/views/h5/H5TaskEdit.vue</code>
<code>/h5/todos</code>	<code>h5-todos</code>	<code>@/views/h5/H5TodoList.vue</code>
<code>/h5/todos/new</code>	<code>h5-todo-new</code>	<code>@/views/h5/H5TodoEdit.vue</code>
<code>/h5/todos/:id</code>	<code>h5-todo-edit</code>	<code>@/views/h5/H5TodoEdit.vue</code>
<code>/h5/settings</code>	<code>h5-settings</code>	<code>@/views/h5/H5Settings.vue</code>

4.3 路由守卫（beforeEach）

```
router.beforeEach((to, _from, next) => {
  // 1. 进入H5 → 清除 force_pc 标志
  if (to.path.startsWith('/h5')) clearForcePC()

  // 2. 公开路由（login, diag）
  if (to.meta.public) {
    if (已认证 && 非diag) → 跳转首页 或 H5
    else → 放行
    return
  }

  // 3. 受保护路由未认证
  if (!auth.isAuthenticated) {
```

```
    if (H5路由) → 保存 h5_redirect 到 sessionStorage
    → 跳转 login
    return
  }

  // 4. 已认证：设备自适应
  if (移动端 && 非H5 && 非forcePC) → 跳转 /h5/tasks
  if (桌面端 && H5) → 跳转 PC首页

  // 5. 确保存储适配器 userId 正确
  switchUser(auth.userId)
  next()
})
```

守卫类型

守卫	说明
认证守卫	未登录访问受保护路由 → 跳转 /login
设备自适应守卫	移动端访问PC路由 → 重定向 /h5/tasks
强制PC模式	isForcePC() 标志阻止移动端自动跳转
H5重定向恢复	未登录访问H5页面 → 保存路径 → 登录后跳回

5. 状态管理架构（Pinia）

5.1 taskStore — 任务管理

文件: src/stores/task.ts

核心状态字段

状态	类型	说明
tasks	Ref<Task[]>	主任务列表
trash	Ref<DeletedTask[]>	回收站（7天自动清除）
loaded	Ref<boolean>	数据加载标志
reactivateConfirm	Reactive	重新激活确认弹窗状态

关键 Getters

Getter	说明
todoTasks	tasks.filter(t => t.status === 'todo')
inProgressTasks	tasks.filter(t => t.status === 'in_progress')
doneTasks	tasks.filter(t => t.status === 'done')
recentTasks	最近更新的8个任务（按 updatedAt 降序）

Getter

说明

expiringTrash 回收站中3天内将过期的任务

主要 Actions

Action	说明
load() / reload()	加载本地数据到 store
addTask(data)	创建任务，自动生成 ID 和时间戳
updateTask(id, patch)	更新任务，自动设置 inProgressAt/completedAt
deleteTask(id)	已完成任务禁止删除；其余移入回收站
toggleStatus(id)	状态轮转 todo→in_progress→done→todo
requestToggleStatus(id)	已完成→待办需确认；触发 reactivateConfirm
restoreTask(id)	从回收站恢复
permanentDelete(id)	永久删除（记录 tombstone 防止云端回传）
emptyTrash()	一键清空回收站
getHeatmapData(view)	生成年/月/周热力图数据
persist() / persistTrash()	全量保存（仅用于合并场景）
applyRemoteTask() 等4个	处理远端增量同步变更
purgeExpired()	清除7天以上过期回收站项目

设计要点

- **状态轮转:** todo → in_progress → done → todo（闭环）
- **完成时自动时间戳:** 进入 in_progress 自动记录 inProgressAt；进入 done 自动记录 completedAt
- **XP 回调:** 通过 setOnTaskDone() 外部注入完成回调（解耦 growthStore）
- **Tombstone 机制:** 永久删除的任务 ID 记录到 localStorage，防止云端同步时回传

5.2 memoStore — 备忘录

文件: src/stores/memo.ts

核心状态字段

状态	类型	说明
memos	Ref<MemoItem[]>	备忘录列表
searchQuery	Ref<string>	搜索关键词
activeTag	Ref<string>	当前标签筛选

关键 Getters

Getter	说明
filteredMemos	按搜索词+标签双重过滤
pinnedMemos	置顶备忘录（按 sortOrder 升序）
normalMemos	普通备忘录（按 sortOrder 升序）
allTags	所有标签去重排序

主要 Actions

Action	说明
load()	加载+sortOrder迁移+后台云端同步
addMemo(data)	新增，自动计算 sortOrder
updateMemo(id, patch)	更新（标题/内容/标签/置顶/图标）
togglePin(id)	切换置顶状态并重算 sortOrder
removeMemo(id)	删除备忘录
reorderMemo(id, targetIndex)	组内拖拽排序（stride 1000）

特性

- 搜索支持 **标题、内容（去HTML纯文本）、标签**
- 拖拽排序使用 **sortOrder 字段 + stride 1000** 减少级联更新
- 后台自动从Supabase同步

5.3 todoStore — 待办事项

文件: src/stores/todo.ts

核心状态字段

状态	类型	说明
todos	Ref<TodoItem[]>	待办列表

关键 Getters

Getter	说明
activeTodos	仅显示未关联任务（linkedTaskId === null），按 importance 降序+有日期优先排序

主要 Actions

Action	说明
load()	加载+后台云端同步
addTodo(data)	新增待办
updateTodo(id, patch)	更新
removeTodo(id)	删除（已关联任务从列表移除）
markConverted(id, taskId)	标记待办已转换为任务

特性

- 待办转任务后通过 linkedTaskId 关联，从活跃列表隐藏
- 排序优先：高重要性 > 有日期 > 低重要性

5.4 growthStore — 养成系统

文件: src/stores/growth.ts

核心状态字段

状态	类型	说明
state	Ref<GrowthState>	养成主状态（等级/XP/连续天数/日状态）
xpEvents	Ref<XpEvent[]>	XP 获得事件日志
unlockedAchievements	Ref<AchievementRecord[]>	已解锁成就记录

关键 Getters

Getter	说明
level	当前等级
xp / totalXp	当前XP / 累计XP
xpNext	升级所需 XP (level × 20)
xpProgress	升级进度 (0~1)
streakDays / maxStreakDays	当前连续天数 / 历史最高
dailyState	日状态: vitality / withered / recovery
dailyMessage	随机日寄语（基于 dailyState）
unlockedDefs	已解锁成就定义
visibleLockedDefs	可见但未解锁的成就

主要 Actions

Action	说明
load()	加载+刷新日状态+后台云端同步
refreshDailyState()	每日首次刷新：计算连续天数、枯萎/复苏状态
calculateXp(task, streakDays)	计算任务完成XP（base 10 + 优先5 + 凌晨3 + 截止日5 + 连续×2）
applyXp(result, taskId)	应用XP、检查升级
buildAchievementContext(tasks)	构建成就判定上下文
checkAchievements(ctx)	遍历16个成就条件，自动解锁

成就系统（16个）

分类	成就	条件
里程碑	破土/扎根/深根/参天/林海	完成1/50/200/500/1000个任务
连续	春风/夏雨/秋实/岁寒	连续7/30/100/365天
特殊	星光/时钟/巧手/满月	凌晨10个/截止日20个/单日10个/完成率100%
隐藏	枯木逢春/从头再来/星河入梦	不公开条件

日状态系统（烛 • Candle）

vitality → 昨日有活动且非枯萎态
withered → 中断多天
recovery → 从枯萎态恢复（连续1天完成）

与其他Store的依赖

- 依赖 taskStore：读取任务列表构建成就上下文
- 通过 useGrowthIntegration() 建立回调桥接

5.5 aiStore — AI助手

文件: src/stores/ai.ts

核心状态字段

状态	类型	说明
messages	Ref<AiMessage[]>	AI对话消息列表
config	Ref<AiConfig>	AI配置（API URL/Key/Model）
loading	Ref<boolean>	请求中
extraContext	Ref<string>	额外上下文（AI分析特定任务时注入）

主要 Actions

Action	说明
send(content)	发送消息，支持SSE流式响应
buildSystemPrompt()	构建带任务上下文的系统提示
handleToolCalls(rawToolCalls)	处理模型返回的工具调用
executeToolCall(name, args)	执行工具：create_task/list_tasks/update_task/delete_task
confirmAction(msgId)	用户确认危险操作
rejectAction(msgId)	用户拒绝危险操作

Function Calling 工具（4个）

工具名	说明	需确认
create_task	创建任务（title必填+可选priority/dueDate/startTime/tags）	否
list_tasks	查询任务（today/overdue/all/todo/in_progress/done）	否
update_task	修改任务	是
delete_task	删除任务（移入回收站）	是

流式处理流程

```
用户发送消息
→ addUserMessage()
→ fetch(apiUrl, POST, stream:true)
→ SSE EventStream 逐块解析
→ 累积 delta.content
→ 累积 delta.tool_calls
→ finish: 处理工具调用 / 显示文本回复
→ upsertMessagePersist()
```

与其他Store的依赖

- 依赖 taskStore：读取/创建/修改/删除任务，构建系统提示

5.6 weeklyReportStore — 周报

文件: src/stores/weeklyReport.ts

核心状态字段

状态	类型	说明
reports	Ref<WeeklyReport[]>	周报列表
sortedReports	Computed	按周降序排列
currentWeekMonday	Computed	本周一日期

主要 Actions

Action	说明
generateReport (weekStart)	Phase 1: 生成含AI占位HTML
generateAiSummary (weekStart)	Phase 2: 异步调用AI生成总结
deleteReport (id)	删除报告

周报HTML结构（5大块）

- 00 AI 智能总结（generating/success/failed）
- 01 统计概览（完成率/完成任务/新增任务/XP/连续天数/新增待办）
- 02 本周完成任务（表格：名称/优先级/实际开始时间/耗时/完成日期）
- 03 本周新增待办（列表）
- 04 待完成任务・下周跟进（按截止日期排序）

与其他Store的依赖

- 依赖 taskStore：读取任务列表
- 依赖 todoStore：读取待办列表
- 依赖 aiStore：调用AI生成周报总结

5.7 authStore — 认证

文件: src/stores/auth.ts

核心状态字段

状态	类型	说明
user	Ref<User \ null>	当前用户
hasSession	Ref<boolean>	session是否存在（响应式桥接localStorage）
loading	Ref<boolean>	加载状态
error	Ref<string>	错误消息
verifyCode	Ref<string>	验证码生成值
pendingPhone	Ref<string>	待验证手机号

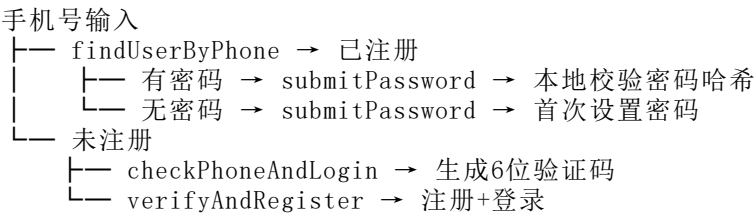
关键 Getters

Getter	说明
isAuthenticated	user !== null \ \ hasSession
userId	user?.id ?? ''

主要 Actions

Action	说明
init()	同步恢复session + 异步拉取Supabase信息
checkPhoneAndLogin(phone)	检查手机号：已注册→直接登录/未注册→生成验证码
verifyAndRegister(code)	验证码校验+注册（任意6位数字）
submitPassword(phone, password, confirm?)	密码登录/注册/设置密码（4种返回状态）
changePassword(old, new)	修改密码（需验证旧密码）
logout()	清除状态+session
changeNickname(nickname)	修改昵称并更新session

认证流程



5.8 timerStore — 计时器

文件: src/stores/timer.ts

状态/Getter	说明
config	{ workStart: '09:00', workEnd: '18:00', workDays: [1..5] }
countdownLabel	动态标签：休息日/距上班/距下班/已下班
countdownMs	距离目标的毫秒数
load() / save()	从存储加载/保存配置

6. 组件体系

6.1 布局组件

AppSidebar

侧边导航栏，包含： - 用户信息（头像/昵称/电话） - 导航菜单（首页/任务/待办/备忘录/AI/养成/周报） - 同步状态显示（在线/同步中/最后同步时间） - 同步日志入口 - 退出登录 - Props: isOnline, syncStatus, lastSyncText, syncLogs - Emits: logout

6.2 任务组件

组件	说明
TaskList	任务主列表（筛选/排序/状态切换/操作菜单）
TaskEditor	任务编辑器（标题/描述/日期/优先级/标签）
TaskEditModal	任务编辑弹窗
TaskDetailModal	任务详情弹窗
TaskRightPanel	任务右侧详情面板
TaskCalendar	任务日历视图
TaskHeatmap	任务热力图（年/月/周视图）
TaskTrendChart	任务趋势图

6.3 备忘录组件

组件	说明
MemoView	左右分栏布局（左侧列表+右侧编辑器）
MemoEditModal	备忘录编辑弹窗（适用于弹窗模式）

6.4 养成系统组件

组件	说明
SpiritWidget	烛灵小组件（等级/XP条/日状态）
SpiritIllustration	烛灵插画
AchievementCard	成就卡片
StreakBadge	连续天数徽章
XpToast	XP获得/升级/成就解锁 Toast提示

6.5 首页组件

组件	说明
GreetingCard	问候卡片（根据时间段问候+日寄语）
CompletionCard	全部完成庆祝卡片
CountdownTimer	上下班倒计时
StatsCards	统计卡片组（今日任务数/完成数等）
TodayProgress	今日进度条

6.6 编辑器组件

组件	说明
RichTextEditor	富文本编辑器（Tiptap全功能，备忘录编辑）
RichTextEditorAsync	异步加载版编辑器（懒加载优化）
MermaidNodeView	Mermaid图表的 VueNodeView 渲染组件
MindmapNodeView	思维导图的 VueNodeView 渲染组件

6.7 通用组件

组件	说明
ConfirmDialog	通用确认弹窗（支持 warning 类型、HTML消息）
SyncLogModal	同步日志查看弹窗
TestReportModal	测试报告弹窗（诊断页）
BackToTop	回到顶部浮动按钮
ReportPoster	周报导出（html2canvas生成图片）
AiChat	AI对话面板
TodoEditModal	待办编辑弹窗

7. 富文本编辑器扩展体系

项目基于 Tiptap v3.26 构建了完整的自定义扩展体系，共 13个扩展。

7.1 扩展全览

扩展名	类型	功能	实现方式
SlashCommand	Plugin	/ 指令面板（16种块类型）	handleTextInput + 原生DOM面板
EmojiExtension	Plugin	: emoji选择器	handleTextInput + 原生DOM网格面板
MemoMention	Mention 配置	[[备忘录引用（双中括号）	@tiptap/extension-mention + suggestion
DoubleBracketLinker	Plugin	[[备忘录引用（备用方案）	handleTextInput检测连续两个[
Mermaid	Node	Mermaid图表块	VueNodeView (MermaidNodeView.vue)
Mindmap	Node	XMind风格思维导图	VueNodeView (MindmapNodeView.vue) + markmap
Callout	Node	提示块（💡 带图标+颜色）	custom Node + wrapIn/toggleWrap

扩展名	类型	功能	实现方式
Toggle	Node	折叠/展开块 (details/summary)	custom Node + addNodeView(DOM操作)
FileAttachment	Node	文件附件内联节点	inline atom Node (base64 data-file)
DragHandle	Plugin	块级拖拽排序	Plugin.view + mousedown/mousemove/mouseup
MarkdownInputRules	Extension	Markdown输入规则 (5个)	InputRule × 5
TrailingNode	Plugin	文档尾部空段落守卫	appendTransaction
TaskItemStylePlugin	Plugin	任务列表项样式注入	Plugin.view update() + inline flex

7.2 SlashCommand Plugin 实现详解

- 触发方式: 块首输入 /
- 搜索过滤: 实时读取 / 后文本, 匹配 label / type
- 面板渲染: 纯DOM操作, CSS分类分组 (基础块/列表/高级块/嵌入)
- 键盘导航: ArrowUp/Down (capture phase事件拦截) + Enter选择 + Escape关闭
- 项选择: mousedown 事件委托在面板容器上
- 查询追踪: requestAnimationFrame 循环读取编辑器内容
- 16个菜单项:

指令	效果
/文本	普通段落
/一级标题 /二级标题 /三级标题	标题
/无序列表 /有序列表 /待办清单	列表
/引用块 /提示块 /折叠块 /分割线	高级块
/代码块 /表格	代码/表格
/Mermaid图表	流程图/时序图/甘特图
/思维导图	XMind风格导图
/链接备忘	备忘录交叉引用

7.3 VueNodeView 实现 (Mermaid / Mindmap)

两个图表扩展使用 @tiptap/vue-3 的 VueNodeViewRenderer 将 Vue 组件嵌入编辑器:

```
// Mermaid.ts
addNodeView() {
  return VueNodeViewRenderer(MermaidNodeView as any)
}

// Mindmap.ts
addNodeView() {
```



```

    return VueNodeViewRenderer(MindmapNodeView as any)
  }

```

特性： - **atom node**（不可编辑内容），isolating: true（隔离选区） - **selectable + draggable**: 可选中，可与DragHandle配合拖拽 - **属性**: code, width, height - **Mermaid**: 使用 mermaid.js 通过 mermaid.render() 生成SVG - **Mindmap**: 使用 markmap-lib 将 Markdown 转为SVG思维导图，支持缩放/平移/折叠

7.4 MarkdownInputRules — 5个快捷输入规则

输入	转换
- 或 * 块首	无序列表
1. 块首	有序列表
[] 或 [x] 块首	待办清单（支持勾选状态）
::: 块首	提示块 (Callout)
>> 块首	折叠块 (Toggle)

7.5 图片/文件附件处理

- **FileAttachment**: 自定义 inline atom 节点，base64编码存储在 data-file 属性
- **图片**: 使用官方 @tiptap/extension-image + tiptap-extension-resize-image 缩放能力
- **DOMPurify**: 对所有HTML内容进行XSS净化

8. Composables 体系

8.1 useTheme — 主题管理

文件: src/composables/useTheme.ts

5种主题模式： | 模式 | data-theme | 说明 | |----|-----|-----| | tencent | tencent | 腾讯蓝（TDesign #0052D9） — **默认** | | dark | dark | 暗黑模式 | | auto | light/dark | 跟随系统（prefers-color-scheme） | | zuru | zuru | ZURU品牌红白灰 | | light | — | 已隐藏：遗留值自动迁移为 tencent |

提供方法： - setTheme(mode) — 设置并持久化到 localStorage cleannotes_theme - toggle() — 轮转 tencent → dark → auto → tencent - 响应式暴露: mode, isDark, isZuru, isTencent

8.2 useSync — 数据同步

文件: src/composables/useSync.ts

- **增量同步**: 每30秒从Supabase拉取远端变更
- **变更应用**: applyRemoteTask() / applyRemoteDeletedTask() / applyRemoteAiMessage() 等

- **同步日志:** pushSyncLog 记录每次同步状态
- **健康检查:** startHealthCheck() 配合 stopHealthCheck()
- **暴露:** isOnline, syncStatus, lastSyncAt, formatLastSync, syncNow
- **停止:** stopAllSync() 退出登录时调用

8.3 useGrowthIntegration — 养成系统桥接

文件: src/composables/useGrowthIntegration.ts

任务完成 → 养成系统的回调桥:

```
taskStore.toggleStatus('done')
  → onTaskDoneCallback(task)
    → growthStore.calculateXp(task, streakDays) # 计算XP
    → growthStore.applyXp(result, task.id)      # 应用XP
    → 检查升级 → showLevelUpToast()
    → growthStore.buildAchievementContext()      # 构建成就上下文
    → growthStore.checkAchievements()           # 检查成就
    → showAchievementToast()
    → showXpToast()
```

8.4 useCompletionCelebration — 完成庆祝

文件: src/composables/useCompletionCelebration.ts

- 监听 todayTotal === todayDone 触发庆祝卡片
- 每天仅显示一次 (localStorage 记录日期)
- 600ms 防抖: 避免同步导致的状态波动误触发
- 按 userId 隔离 localStorage 键

8.5 useH5Data — H5数据桥接

文件: src/composables/useH5Data.ts

- 直连Supabase, 无本地缓存
- 提供完整的任务/待办 CRUD
- toggleTaskStatus() 直接操作云端并返回更新后数据
- convertTodoToTask() 待办转任务 (同时创建任务+标记待办linkedTaskId)

9. 构建配置

9.1 Vite 配置详解

文件: vite.config.ts

```
defineConfig({
  base: './', // 相对路径 (Electron/Tauri兼容)
  define: {
    __APP_VERSION__: '1.3.0', // 构建时注入版本号
    __BUILD_TIME__: '2026-07-10 12:00', // 构建时间
  },
  plugins: [
    vue(), // Vue3 SFC编译
    tailwindcss(), // @tailwindcss/vite 插件
    copy-readme, // 构建时复制README→public/readme.txt
    visualizer() (ANALYZE=true时) // Bundle分析
  ],
  resolve: { alias: { '@': './src' } },
  server: {
    port: 1420, // Tauri开发端口
    strictPort: true,
    hmr: host ? { protocol: 'ws', host, port: 1421 } : undefined,
    watch: { ignored: ['**/src-tauri/**'] }, // Tauri后端文件监听
  },
  build: {
    rollupOptions: {
      output: {
        manualChunks: { /* 代码分割 */ }
      }
    }
  }
})
```

9.2 环境变量 (编译时注入)

变量	来源	示例值
__APP_VERSION__	package.json version	"1.3.0"
__BUILD_TIME__	构建时刻	"2026-07-10 14:30"

在 TypeScript 中使用前需先声明:

```
declare const __APP_VERSION__: string
declare const __BUILD_TIME__: string
```

9.3 代码分割策略 (manualChunks)

Chunk	内容	触发条件
vendor-prosemirror	prosemirror 核心库	node_modules中包含prosemirror
vendor-tiptap-table	table/row/header/cell 扩展	表格功能
vendor-tiptap-task	task-list / task-item 扩展	任务清单功能
vendor-tiptap-mention	mention / suggestion 扩展	提及功能
vendor-tiptap-core	其余 @tiptap 包 + resize-image	Tiptap核心+其他
vendor-tiptap-custom	src/extensions/ 自定义扩展	自定义编辑器扩展
默认	其他 node_modules	Vite自动处理

策略特点: 按**功能组**划分而非按包名, 实现更细致的懒加载。

9.4 TailwindCSS v4 配置

- 使用 @tailwindcss/vite 插件（v4集成方式）
- style.css 首行 @import "tailwindcss"
- 主题色通过 **CSS变量系统** 定义（4套主题），取代 Tailwind 传统 theme 配置
- 所有颜色使用语义变量：var(--color-primary), var(--color-bg-1) 等

9.5 构建命令

命令	说明
npm run dev	开发模式（vite --host, port 1420）
npm run build	生产构建（先 vue-tsc 类型检查）
npm run build:gh-pages	GitHub Pages 部署构建（base=/cleannotes-prod/）
npm run preview	预览生产构建
npm run start	启动预览服务器（port 4173）
npm run copy:prod	复制构建产物到生产目录

10. 工具函数

10.1 time.ts — 时间处理

文件: src/utils/time.ts

函数	返回	说明
toUTCISO(date?)	string	生成UTC ISO时间戳（带Z）：2026-07-01T04:34:00.000Z
toLocalISO(date?)	string	生成本地ISO（无时区）— 仅用于 datetime-local 输入
toLocalDate(date?)	string	生成本地日期 YYYY-MM-DD
normalizeTimestamp(ts)	string null	将旧格式本地时间转为UTC ISO（修复历史bug）

约定: 所有 Supabase 时间戳存储使用 UTC ISO + Z 后缀。

10.2 crypto.ts — 密码哈希

文件: src/utils/crypto.ts

基于浏览器原生 Web Crypto API 的零依赖密码方案：

函数	说明
generateSalt()	生成16字节随机盐（base64编码）

函数	说明
hashPassword(password, salt)	PBKDF2-HMAC-SHA256, 迭代210,000次
verifyPassword(password, salt, hash)	恒定时间比较 (防止时序攻击)

安全参数: - PBKDF2-HMAC-SHA256 - 迭代次数: 210,000 (OWASP 2023推荐) - 盐长度: 16字节 (128位随机) - 派生密钥长度: 256位

10.3 markdownExport.ts — 导出

文件: src/utils/markdownExport.ts

函数	说明
htmlToMarkdown(html)	HTML富文本 → Markdown (完整支持)
htmlToPlainText(html)	HTML → 纯文本 (保留换行)

支持的格式转换: - 标题 h1-h3、段落、列表 (有序/无序/任务清单) - 引用、代码块、分割线 - Callout提示块、Toggle折叠块 - 表格 (含表头分隔线) - 内联格式: 加粗/斜体/删除线/行内代码/链接/图片 - 文件附件、备忘录引用 ([[名称]])

10.4 device.ts — 设备检测

文件: src/utils/device.ts

函数	说明
isMobileDevice()	检测移动设备 (UA关键词 + 屏幕宽度 ≤ 480)
setForcePC()	用户手动切换桌面版, 阻止移动端自动跳转
clearForcePC()	进入H5页面时清除强制PC标记
isForcePC()	检查是否已强制桌面版模式

UA检测关键词: Android|webOS|iPhone|iPad|iPod|BlackBerry|IEMobile|Opera Mini|Mobile

11. 类型系统总览

文件: src/types/index.ts

类型	用途	关键字段
Task	任务实体	id, title, status, priority, dueDate, startTime, startDate, tags, createdAt, updatedAt, completedAt, inProgressAt
DeletedTask	回收站任务	extends Task + deletedAt

类型	用途	关键字段
TodoItem	待办事项	id, title, description, estimatedStart, estimatedEnd, linkedTaskId, importance
MemoItem	备忘录	id, title, content, tags, pinned, icon, sortOrder
User	用户	id, phone, nickname
Session	登录会话	userId, phone, nickname, loginAt, expiresAt
AiMessage	AI消息	id, role, content, timestamp, pendingAction?
AiConfig	AI配置	apiUrl, apiKey, model
AiPendingAction	待确认操作	toolCallId, toolName, args, description, confirmed
GrowthState	养成状态	level, xp, totalXp, streakDays, maxStreakDays, dailyState, witheredDays
XpEvent	XP事件	id, taskId, source, xp
AchievementDef	成就定义	id, name, description, category, condition, isHidden
AchievementRecord	成就记录	id, unlockedAt
WeeklyReport	周报	id, weekStart, weekEnd, content, summary, aiSummary?, aiSummaryStatus?
WeeklyReportSummary	周报摘要	tasksCreated, tasksCompleted, todosCreated, totalXpGained, completionRate, streakDays
TimerConfig	计时配置	workStart, workEnd, workDays
HeatmapCell	热力图单元格	date, count, level (0-4)
HeatmapView	热力图视图	'year' 'month' 'week'
DailyState	日状态	'vitality' 'withered' 'recovery'
TaskStatus	任务状态	'todo' 'in_progress' 'done'
TaskPriority	任务优先级	'low' 'medium' 'high'
XpSource	XP来源	'complete' 'priority' 'night' 'deadline' 'streak' 'achievement'
AchievementCategory	成就分类	'milestone' 'streak' 'special' 'hidden'

12. CSS 主题系统

12.1 4套主题方案

通过 `data-theme` 属性切换，全部使用 CSS 变量（无运行时JS计算）：

主题	data-theme	主色调	适用场景
腾讯蓝	tencent	#0052D9 (TDesign蓝)	默认主题
暗黑	dark	#6b81f8	夜间模式
ZURU	zuru	#CB312D (品牌红)	ZURU内部使用
自动	auto → light/dark	跟随系统	自动适配

12.2 CSS 变量清单

语义色: `--color-primary`, `--color-primary-light`, `--color-primary-hover`, `--color-primary-disabled` `--color-success/light/lighter/text`, `--color-warning/light/text`, `--color-danger/light/text` `--color-info/light/lighter/text`, `--color-accent/light/text`

文字色: `--color-text-1` ~ `--color-text-4` （从深到浅）

背景色: `--color-bg-1` ~ `--color-bg-4`, `--color-surface`

边框: `--color-border`, `--color-border-light`

效果: `--color-focus-ring`, `--color-shadow/md`, `--color-overlay/md`

代码: `--color-code-bg/text`, `--color-pre-bg/text`

13. 数据持久化架构

13.1 存储适配器模式

```
getStorage()  
├─ Supabase可用 → supabaseAdapter（云端存储）  
└─ Supabase不可用 → localAdapter（localStorage, format: cleannotes_{userId}_{type}）
```

所有 Store 通过 `getStorage()` 统一获取当前存储后端，无需关心底层实现。

13.2 混合同步策略（hybrid.ts）

写入时：
Store → upsert → Supabase（primary）+ localStorage（backup）

读取时：

```
Store ← localStorage (instant) + Supabase (background merge)
```

同步时：

```
30秒增量拉取 → fetchRemoteChanges()
  → 比较 updatedAt / deletedAt
  → 更新本地 (applyRemoteTask / applyRemoteDeletedTask 等)
  → 只写 localStorage，不写 Supabase (避免循环)
```

13.3 Tombstone 墓碑机制

永久删除的任务 ID 记录在内存 Set 中（持久化到 localStorage），防止远端同步时回传已删除数据。

13.4 时间迁移（v5）

- 历史数据可能有旧格式时间戳（无Z后缀或无时区）
- runTimeMigration() 在 App.vue onMounted 时执行
- 迁移标记 cleannotes_time_migrated_v5 防止重复执行
- normalizeTimestamp() 在 Supabase 同步时自动修复旧格式

14. 关键设计决策

决策	理由
Hash路由模式	兼容静态部署（无服务端配置要求）
Pinia Setup Style	与 Composition API 风格一致
CSS变量主题而非CSS-in-JS	零运行时开销，一套变量多套皮肤
Tiptap自定义扩展（非suggestion库）	更精细的DOM控制，避免库约束
每30秒增量同步（非实时推送）	简单可靠，无WebSocket复杂度
localStorage + Supabase双写	离线可用+云端备份
密码客户端哈希（PBKDF2）	明文不出客户端，安全合规
UTC+Z时间戳存储	跨时区一致性
手动代码分割（manualChunks）	精确控制加载粒度
mutation observer禁用	利用PM plugin.view.update() 在正确时机操作DOM